

### UNIT 3:

**Transaction Management and Concurrency Control:-** Transaction: Evaluating Transaction Results, Transaction Properties, Transaction Management with SQL, The Transaction Log – Concurrency Control: Lost Updates, Uncommitted Data, Inconsistent Retrievals, The Scheduler – Concurrency Control with Locking Methods: Lock Granularity, Lock Types, Two Phase Locking to Ensure Serializability, Deadlocks – Concurrency Control with Time stamping Methods: Wait/Die and Wait/Wound Schemes – Concurrency Control with Optimistic Methods - Database Recovery Management: Transaction Recovery.

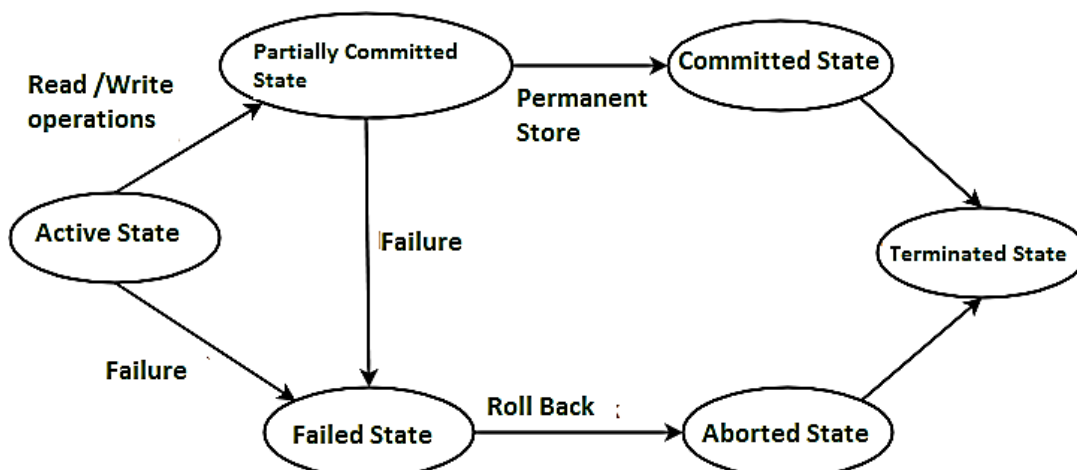
### Transaction Management and Concurrency Control

Transactions are a set of operations used to perform a logical set of work. A transaction usually means that the data in the database has changed. One of the major uses of DBMS is to protect the user data from system failures. It is done by ensuring that all the data is restored to a consistent state when the computer is restarted after a crash. The transaction is any one execution of the user program in a DBMS. One of the important properties of the transaction is that it contains a finite number of steps. Executing the same program multiple times will generate multiple transactions.

**A transaction can include the following basic database access operation.**

- **Read/Access data (R):** Accessing the database item from disk (where the database stored data) to memory variable.
- **Write/Change data (W):** Write the data item from the memory variable to the disk.
- **Commit:** Commit is a transaction control language that is used to permanently save the changes done in a transaction

### Transaction States



## A transaction moves through various states:

- **Active:** The transaction is being executed.
- **Partially Committed:** The transaction has completed its execution but has not yet been committed.
- **Committed:** All operations are successfully completed, and changes are permanent.
- **Aborted:** The transaction is terminated due to some error or failure, and all changes are rolled back.

## Properties of Transactions: ACID

The ACID properties define the behaviour of a transaction:

### ➤ Atomicity:

- A transaction is atomic, meaning it is indivisible. It either **completes entirely** or **does not start at all**. If a transaction cannot be completed, all changes are rolled back, and the database is left in the same state as before the transaction began.

### ➤ Consistency:

- A transaction moves the database from one consistent state to another. It ensures that **any transaction** takes the database from a valid state to another valid state, adhering to all defined constraints, rules, and integrity checks.

### ➤ Isolation:

- Transactions are isolated from each other. The intermediate state of a transaction is not visible to others. Even though transactions may run concurrently, each one should not interfere with others, providing **logical isolation**.

### ➤ Durability:

- Once a transaction is committed, its effects are permanent, even in the event of a system failure. The changes are stored in non-volatile memory.

## Example: Bank Account Transfer

Consider a scenario where you want to transfer money from **Account A** to **Account B** in a bank database.

### 1. Initial state:

- Account A: \$500
- Account B: \$300

### 2. Transaction:

- **Step 1:** Subtract \$100 from **Account A**.
- **Step 2:** Add \$100 to **Account B**.

The **transaction** is supposed to:

1. Decrease the balance of **Account A** by \$100.
2. Increase the balance of **Account B** by \$100.

## ACID Properties Applied to the Transaction:

### • Atomicity:

- The transaction will **either fully complete** (both operations succeed) or **not execute at all** (if there is an error in any step). For example, if Step 1 (subtracting from Account A) is successful, but Step 2 (adding to Account B) fails, the transaction will **rollback**, meaning Account A will still have \$500, and Account B will still have \$300.

- **Consistency:**
  - The database ensures that at the beginning and end of the transaction, the state of the database is consistent. After a transfer, the total money in both accounts should be the same (i.e.,  $\$500 + \$300 = \$500 + \$300$ ), and no money is "lost" or "created."
- **Isolation:**
  - If two users try to transfer money at the same time, their actions should not interfere with each other. If user A is transferring money from Account A to Account B while user B is transferring money from Account C to Account D, the transactions should not overlap and cause incorrect results (e.g., both transactions might affect the same account).
- **Durability:**
  - Once the transaction is successfully committed, the changes are permanent. Even if the system crashes immediately after committing the transaction, the money is transferred and stored correctly in both accounts.

## SQL Transaction

A transaction in SQL is a sequence of one or more SQL statements executed as a **single unit of work**. These statements could be performing operations like **INSERT**, **UPDATE**, or **DELETE**. The main idea behind a transaction is that all changes made within it should either be completely successful or fully undone in case of failure. In the case of an error, a **rollback** ensures that no partial changes are saved.

### The SQL Transaction Process

Transactions are controlled using specific SQL commands. The most common commands used for managing transactions are:

1. **BEGIN TRANSACTION:** Starts a transaction.
2. **COMMIT:** Commits the transaction, saving all changes made.
3. **ROLLBACK:** Rolls back the transaction, undoing all changes made since the **BEGIN TRANSACTION**.
4. **SAVEPOINT:** Marks a point within a transaction to which you can later roll back.

### Transaction Control Commands

| Command                      | Purpose                                                        | Example                                    |
|------------------------------|----------------------------------------------------------------|--------------------------------------------|
| <b>BEGIN TRANSACTION</b>     | Starts a new transaction.                                      | <b>BEGIN TRANSACTION</b><br>TransferFunds; |
| <b>COMMIT</b>                | Saves all changes made during the transaction.                 | <b>COMMIT</b> ;                            |
| <b>ROLLBACK</b>              | Undoes changes made during the transaction.                    | <b>ROLLBACK</b> ;                          |
| <b>SAVEPOINT</b>             | Creates a point in the transaction to which you can roll back. | <b>SAVEPOINT SP1</b> ;                     |
| <b>ROLLBACK TO SAVEPOINT</b> | Rolls back the transaction to a specific savepoint.            | <b>ROLLBACK TO SP1</b> ;                   |
| <b>RELEASE SAVEPOINT</b>     | Removes a savepoint from the transaction.                      | <b>RELEASE SAVEPOINT SP2</b> ;             |

## BEGIN TRANSACTION Command

The **BEGIN TRANSACTION** command marks the beginning of a new transaction. All SQL statements that follow this command will be part of the same transaction until a **COMMIT** or **ROLLBACK** is encountered. This command doesn't make any changes to the database, it just starts the transaction.

### Syntax:

*BEGIN TRANSACTION transaction\_name ;*

### Example

BEGIN TRANSACTION TransferFunds;

## COMMIT Command

The **COMMIT** command is used to save all changes made during the current transaction to the database. Once a transaction is committed, the changes are permanent.

### Syntax:

*COMMIT;*

Here is the sample **Student** table that will be used to perform the operations in this example. This table contains basic student details such as **ID**, **name**, **age**, and other relevant information that will be manipulated using various **transaction control commands**.

| Student |        |         |            |     |
|---------|--------|---------|------------|-----|
| Rol_No  | Name   | Address | Phone      | Age |
| 1       | Ram    | Delhi   | 9455123451 | 18  |
| 2       | Ramesh | Gurgaon | 9652431543 | 18  |
| 3       | Sujit  | Rohtak  | 9156253131 | 20  |
| 4       | Suresh | Delhi   | 9156768971 | 18  |
| 3       | Sujit  | Rohtak  | 9156253131 | 20  |
| 2       | Ramesh | Gurgaon | 9652431543 | 18  |

### Example

Following is an example which would delete those records from the table which have age = 20 and then **COMMIT** the changes in the database.

DELETE FROM Student WHERE AGE = 20;  
COMMIT;

| Rol_No | Name   | Address | Phone      | Age |
|--------|--------|---------|------------|-----|
| 1      | Ram    | Delhi   | 9455123451 | 18  |
| 2      | Ramesh | Gurgaon | 9652431543 | 18  |
| 4      | Suresh | Delhi   | 9156768971 | 18  |
| 2      | Ramesh | Gurgaon | 9652431543 | 18  |

## ROLLBACK Command

The ROLLBACK command is used to undo all changes made in the current transaction. It is used when an error occurs or when the desired changes cannot be completed. The database will revert to the state it was in before the BEGIN TRANSACTION was executed.

### Syntax:

```
ROLLBACK;
```

### Example

Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database. In this case, the DELETE operation is undone, and the changes to the database are not saved.

```
DELETE FROM Student WHERE AGE = 20;  
ROLLBACK;
```

| Student |        |         |            |     |
|---------|--------|---------|------------|-----|
| Rol_No  | Name   | Address | Phone      | Age |
| 1       | Ram    | Delhi   | 9455123451 | 18  |
| 2       | Ramesh | Gurgaon | 9652431543 | 18  |
| 3       | Sujit  | Rohtak  | 9156253131 | 20  |
| 4       | Suresh | Delhi   | 9156768971 | 18  |
| 3       | Sujit  | Rohtak  | 9156253131 | 20  |
| 2       | Ramesh | Gurgaon | 9652431543 | 18  |

## SAVEPOINT Command

A SAVEPOINT is used to create a checkpoint within a transaction. We can roll back to a specific SAVEPOINT instead of rolling back the entire transaction. This allows us to **undo part** of the transaction **rather than** the entire transaction.

### Syntax:

```
SAVEPOINT SAVEPOINT_NAME;
```

### Example

```
SAVEPOINT SP1;  
    //Savepoint created.  
DELETE FROM Student WHERE AGE = 20;  
    //deleted  
SAVEPOINT SP2;  
    //Savepoint created.
```

| Rol_No | Name   | Address | Phone      | Age |
|--------|--------|---------|------------|-----|
| 1      | Ram    | Delhi   | 9455123451 | 18  |
| 2      | Ramesh | Gurgaon | 9652431543 | 18  |
| 4      | Suresh | Delhi   | 9156768971 | 18  |
| 2      | Ramesh | Gurgaon | 9652431543 | 18  |

### Explanation:

From the above example Sample table1, Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database by keeping Savepoints. Here SP1 is first SAVEPOINT created before deletion. In this example one deletion have taken place. After deletion again SAVEPOINT SP2 is created.

### ROLLBACK TO SAVEPOINT

The ROLLBACK TO SAVEPOINT command allows us to roll back the transaction to a specific savepoint, effectively undoing changes made after that point.

#### Syntax:

```
ROLLBACK TO SAVEPOINT SAVEPOINT_NAME;
```

#### Example

Deletion have been taken place, let us assume that we have changed our mind and decided to ROLLBACK to the SAVEPOINT that we identified as SP1 which is before deletion. So, In this case the DELETE operation is undone, and the transaction is returned to the state it was in at the SP1 savepoint.

```
ROLLBACK TO SP1;
```

```
//Rollback completed
```

| Student |        |         |            |     |
|---------|--------|---------|------------|-----|
| Rol_No  | Name   | Address | Phone      | Age |
| 1       | Ram    | Delhi   | 9455123451 | 18  |
| 2       | Ramesh | Gurgaon | 9652431543 | 18  |
| 3       | Sujit  | Rohtak  | 9156253131 | 20  |
| 4       | Suresh | Delhi   | 9156768971 | 18  |
| 3       | Sujit  | Rohtak  | 9156253131 | 20  |
| 2       | Ramesh | Gurgaon | 9652431543 | 18  |

### RELEASE SAVEPOINT Command

This command is used to remove a SAVEPOINT that we have created. Once a SAVEPOINT has been released, we can no longer use the ROLLBACK command to undo transactions performed since the last SAVEPOINT. It is used to initiate a database transaction and used to specify characteristics of the transaction that follows.

#### Syntax:

```
RELEASE SAVEPOINT SAVEPOINT_NAME
```

#### Example

Once the savepoint SP2 is released, we can no longer roll back to it.

```
RELEASE SAVEPOINT SP2; -- Release the second savepoint.
```

### Types of SQL Transactions

There are different types of transactions based on their nature and the specific operations they perform:

- **Read Transactions:** Used to only read the data, typically with SELECT queries.
- **Write Transactions:** These involve modifying the data in the database with INSERT, UPDATE, or DELETE operations.

- **Distributed Transactions:** These transactions span multiple databases and ensure consistency across them.
- **Implicit Transactions:** Automatically started by SQL Server for certain operations.
- **Explicit Transactions:** Manually controlled transactions where the user begins and ends the transaction using BEGIN TRANSACTION, COMMIT, and ROLLBACK.

## Transaction Log in DBMS

A **Transaction Log** is a crucial component in a **Database Management System (DBMS)** that records all changes made to the database during the execution of transactions. The transaction log ensures the **ACID properties** (Atomicity, Consistency, Isolation, Durability) are upheld, and it provides the necessary information to **recover** the database to a consistent state in case of failures like crashes.

### Key Functions of the Transaction Log

- **Recording Changes:** Every time a transaction modifies the database (such as an insert, update, or delete), the **transaction log** records the details of that modification. This includes the old values (before the modification) and new values (after the modification).
- **Crash Recovery:** In the event of a system crash, the transaction log helps to **recover** the database to a consistent state by redoing or undoing transactions based on the logged information.
- **Ensuring Durability:** By recording every change, the transaction log ensures that once a transaction is committed, its changes are permanent, even if the system crashes immediately afterward.
- **Supporting Rollback:** If a transaction is aborted or if an error occurs, the log allows the system to **rollback** the transaction, undoing the changes made during that transaction.

### Components of a Transaction Log

The **transaction log** typically includes the following components:

- **Transaction ID:** A unique identifier for each transaction.
- **Operation Type:** The type of operation (INSERT, UPDATE, DELETE).
- **Before Image:** The state of the data **before** the transaction was applied (for rollback purposes).
- **After Image:** The state of the data **after** the transaction was applied (for recovery purposes).
- **Timestamp:** The time when the operation occurred.
- **Log Sequence Number (LSN):** A unique identifier for each log entry, which helps in ensuring that logs are applied in the correct order during recovery.

### Structure of a Transaction Log Entry

Each entry in the transaction log typically contains the following:

- **Transaction ID:** Unique identifier for the transaction.
- **Type of Operation:** Whether it's an INSERT, UPDATE, or DELETE.
- **Data Affected:** The data modified (e.g., the row or field that was changed).
- **Before Image:** The value before the modification (for undoing the change in case of a rollback).



- **After Image:** The value after the modification (for redoing the change if needed during recovery).
- **Timestamp:** When the transaction took place.
- **Transaction Status:** Whether the transaction was committed or rolled back.

### Transaction Log Example

Let's say a transaction is updating a bank account's balance. Here's how the transaction log might look:

#### Transaction Log Entry (for updating Account A's balance):

| Transaction ID | Operation Type | Table    | Row ID | Before Image | After Image | Timestamp           | Log Sequence Number (LSN) |
|----------------|----------------|----------|--------|--------------|-------------|---------------------|---------------------------|
| T1             | UPDATE         | Accounts | A1     | \$500        | \$400       | 2025-03-04 10:00:00 | 1001                      |
| T1             | UPDATE         | Accounts | A2     | \$300        | \$400       | 2025-03-04 10:00:00 | 1002                      |

- **Transaction ID (T1):** Identifies the transaction.
- **Operation Type:** An UPDATE operation was performed.
- **Table:** The operation was performed on the **Accounts** table.
- **Row ID:** The rows in the **Accounts** table being updated.
- **Before Image:** The old values of the account balance before the transaction.
- **After Image:** The new values after the transaction.
- **Timestamp:** The time at which the operation occurred.
- **LSN:** Log Sequence Number, a unique identifier for each log entry.

## Concurrency Control in DBMS

In a database management system (DBMS), allowing transactions to run concurrently has significant advantages, such as better system resource utilization and higher throughput. However, it is crucial that these transactions do not conflict with each other. The ultimate goal is to ensure that the database remains consistent and accurate. For instance, if two users try to book the last available seat on a flight at the same time, the system must ensure that only one booking succeeds. Concurrency control is a critical mechanism in DBMS that ensures the consistency and integrity of data when multiple operations are performed at the same time.

- Concurrency control is a concept in Database Management Systems (DBMS) that ensures multiple transactions can simultaneously access or modify data without causing errors or inconsistencies. It provides mechanisms to handle the concurrent execution in a way that maintains ACID properties.
- By implementing concurrency control, a DBMS allows transactions to execute concurrently while avoiding issues such as deadlocks, race conditions, and conflicts between operations.
- The main goal of concurrency control is to ensure that simultaneous transactions do not lead to data conflicts or violate the consistency of the database. The concept of serializability is often used to achieve this goal.



## Concurrent Execution can lead to various challenges:

- **Dirty Reads:** One transaction reads uncommitted data from another transaction, leading to potential inconsistencies if the changes are later rolled back.
- **Lost Updates:** When two or more transactions update the same data simultaneously, one update may overwrite the other, causing data loss.
- **Inconsistent Reads:** A transaction may read the same data multiple times during its execution, and the data might change between reads due to another transaction, leading to inconsistency.

## Lost Updates

### Problem:

- A **lost update** occurs when two transactions simultaneously update the same data, and one of the updates is lost because it was overwritten by the other transaction's update.

**Example:** Consider two transactions, **T1** and **T2**, both trying to update the same bank account.

- **Account Balance:** \$500
- **Transaction T1:** Withdraws \$100 from the account.
- **Transaction T2:** Withdraws \$150 from the account.

### *Scenario without Concurrency Control:*

1. **T1** reads the account balance, which is \$500.
2. **T2** also reads the account balance, which is still \$500.
3. **T1** writes the updated balance, \$400 (500 - 100).
4. **T2** writes the updated balance, \$350 (500 - 150).

As a result, **T1's update is lost**, and the account balance becomes \$350 instead of the correct value of \$400 after both transactions should have been processed. This is an example of a **lost update**.

### Solution:

- **Locking:** To prevent lost updates, **locks** are used. When **T1** locks the account balance, **T2** will have to wait until **T1** completes its transaction before it can access the same data.
- **Isolation:** Isolation levels like **Serializable** or **Repeatable Read** ensure that transactions do not overwrite each other.

## Uncommitted Data (Dirty Read)

### Problem:

- An **uncommitted data** (also called **dirty read**) problem occurs when a transaction reads data that has been modified by another transaction but has not yet been committed. If the second transaction is rolled back, the first transaction will have read data that was never valid.

**Example:** Consider two transactions, **T1** and **T2**, interacting with the same account.

- **Account Balance:** \$500
- **Transaction T1:** Withdraws \$100 from the account.
- **Transaction T2:** Reads the balance of the account.

### Scenario:

1. **T1** begins and withdraws \$100, but **T1** hasn't committed yet.
2. **T2** reads the account balance as **\$400** (since T1 has modified it but hasn't committed).
3. **T1** then encounters an error and **rolls back** the transaction.

4. **T2** still sees the balance as \$400, even though this data was never actually committed, leading to **dirty read**.

This is problematic because **T2** read data that **T1** has not committed, and **T2** may act on invalid information.

#### **Solution:**

- **Read Committed Isolation Level:** Ensures that a transaction will only read data that has been committed. This prevents **dirty reads**.
- **Locking:** A **shared lock** can be placed on data during reads, preventing other transactions from making changes while the data is being read.

### **Inconsistent Retrievals (Non-Repeatable Reads)**

#### **Problem:**

- **Inconsistent retrievals** (also known as **non-repeatable reads**) occur when a transaction reads the same data multiple times, and the data has changed between reads. This can lead to inconsistencies in results within the same transaction.

**Example:** Consider two transactions, **T1** and **T2**, both reading and updating the same account.

- **Account Balance:** \$500
- **Transaction T1:** Reads the balance of the account, updates it.
- **Transaction T2:** Also updates the account balance.

#### **Scenario:**

1. **T1** reads the account balance, which is \$500.
2. **T2** updates the balance to \$600.
3. **T1** reads the balance again, which is now \$600, but **T1** expected the value to still be \$500. This is an **inconsistent retrieval** since **T1** saw two different values for the same account within the same transaction.

#### **Solution:**

- **Repeatable Read Isolation Level:** Ensures that once a transaction reads a value, it will see the same value for subsequent reads, preventing **non-repeatable reads**.
- **Locks:** Using **shared locks** on data while reading can prevent other transactions from modifying it until the current transaction is complete.

### **The Scheduler**

A **scheduler** is a component of a DBMS that controls the execution order of transactions, ensuring they are executed in a way that maintains the consistency and isolation of the database. It does this by managing the **locking** of data, ordering transactions, and enforcing concurrency control protocols.

The scheduler determines the **serializability** of the schedule (the order in which transactions are executed) to ensure that the database ends in a consistent state.

- **Serial Schedule:** A schedule where transactions are executed one after the other, without overlapping.
- **Non-serial Schedule:** A schedule where transactions are interleaved, but it needs to ensure that the final result is equivalent to some serial schedule.

**Example of a Scheduler:** Consider two transactions, **T1** and **T2**, where:

- **T1** wants to **read** and **update** Account A.
- **T2** wants to **read** and **update** Account B.

### Scenario without Scheduler:

- **T1** and **T2** may both try to access the accounts at the same time. This could lead to issues like lost updates or inconsistent data.

### Scheduler in Action:

- The scheduler ensures that the transactions are executed in a **serializable order**. For example, it might schedule **T1** to execute fully before **T2**, or **T2** before **T1**, depending on the isolation level and locking policies.

The **scheduler** ensures that the database maintains **serializability** (correctness) even when transactions are interleaved. A **serializable schedule** ensures that even though transactions are executed concurrently, the outcome is the same as if they were executed serially (one at a time).

## Concurrency Control with Locking Methods: Lock Granularity, Lock Types, Two Phases Locking to Ensure Serializability, Deadlocks

Concurrency control in database management systems (DBMS) is essential for ensuring data integrity and consistency when multiple transactions are executed simultaneously. Here's a breakdown of key concepts related to locking methods, including lock granularity, types of locks, two-phase locking (2PL), and deadlocks.

### Lock Granularity

Lock granularity refers to the size of the data that a lock protects. It can be categorized into three types:

- **Coarse-Grained Locking:** Locks larger data units (e.g., an entire table). This approach reduces overhead but can lead to higher contention, as multiple transactions may be blocked from accessing different rows in the same table.
- **Fine-Grained Locking:** Locks smaller data units (e.g., individual rows). This allows for greater concurrency but increases the overhead of managing many locks.
- **Medium-Grained Locking:** A compromise between the two, locking a group of related rows.

### Lock Types

Locks can be classified into several types:

- **Shared Lock (S Lock):** A transaction can read the data but cannot modify it. Multiple transactions can hold a shared lock on the same data.
- **Exclusive Lock (X Lock):** A transaction can both read and write the data. Only one transaction can hold an exclusive lock on a given piece of data at a time.
- **Update Lock (U Lock):** Used when a transaction intends to update a data item. It allows for some level of concurrency while preventing deadlocks.

## Two-Phase Locking (2PL)

Two-Phase Locking is a protocol that guarantees serializability of transactions. It consists of two phases:

- **Growing Phase:** A transaction can acquire locks but cannot release any. This continues until the transaction has acquired all necessary locks.
- **Shrinking Phase:** A transaction can release locks but cannot acquire any new ones. Once a transaction releases a lock, it cannot go back to the growing phase.

### Example of 2PL:

1. Transaction T1 acquires an exclusive lock on data item A.
2. T1 then acquires a shared lock on data item B.
3. T1 performs its operations and releases the locks, entering the shrinking phase.

## Deadlocks

A deadlock occurs when two or more transactions are waiting for each other to release locks, resulting in a standstill. In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a Halt.

### Simple Example of Deadlock:

- **Transaction T1** locks resource A and waits for resource B.
- **Transaction T2** locks resource B and waits for resource A.

This situation creates a deadlock because T1 cannot proceed until T2 releases B, and T2 cannot proceed until T1 releases A.

## Handling Deadlocks

Deadlocks can be managed using various strategies:

- **Deadlock Prevention:** Ensure that a system will never enter a deadlock state, often by structuring resource requests in a way that prevents circular wait.
- **Deadlock Detection:** Periodically check for deadlocks and abort one of the transactions involved to break the deadlock.
- **Deadlock Avoidance:** Use protocols like wait-die or wound-wait to manage transaction ordering.

## Features of Deadlock in a DBMS

1. **Mutual Exclusion:** Each resource can be held by only one transaction at a time, and other transactions must wait for it to be released.
2. **Hold and Wait:** Transactions can request resources while holding on to resources already allocated to them.
3. **No Pre-emption:** Resources cannot be taken away from a transaction forcibly, and the transaction must release them voluntarily.

4. **Circular Wait:** Transactions are waiting for resources in a circular chain, where each transaction is waiting for a resource held by the next transaction in the chain.
5. **Indefinite Blocking:** Transactions are blocked indefinitely, waiting for resources to become available, and no transaction can proceed.
6. **System Stagnation:** Deadlock leads to system stagnation, where no transaction can proceed, and the system is unable to make any progress.
7. **Inconsistent Data:** Deadlock can lead to inconsistent data if transactions are unable to complete and leave the database in an intermediate state.
8. **Difficult to detect and Resolve:** Deadlock can be difficult to detect and resolve, as it may involve multiple transactions, resources, and dependencies.

## Concurrency Control with Time stamping: Wait/Die and Wait/Wound Schemes

Concurrency control in databases ensures that multiple transactions can access and modify shared data concurrently without leading to inconsistencies. One of the ways to achieve concurrency control is by using **timestamp ordering**. In timestamp-based methods, each transaction is assigned a unique timestamp, and the system uses these timestamps to manage the order of operations, ensuring that transactions proceed in a way that maintains database consistency.

Two common **timestamp-based** concurrency control methods are the **Wait/Die** and **Wait/Wound** schemes. These schemes are used to handle **deadlock** and **conflicting transactions** in a way that prevents unwanted situations like **lost updates**, **temporary inconsistency**, or **deadlock**.

### Wait/Die Scheme

- **Wait/Die** is used when a transaction is requesting a resource (like a data item) that is currently held by another transaction. The decision to either allow or block the requesting transaction is based on **timestamps**.
- **If the requesting transaction has a smaller timestamp (i.e., it is older)** than the transaction holding the resource, the requesting transaction **waits**.
- **If the requesting transaction has a larger timestamp (i.e., it is younger)**, the requesting transaction is **killed (dies)**, and it is restarted later with the same timestamp.

**Example:** Consider two transactions, **T1** and **T2**:

- **T1** has a timestamp of 1.
- **T2** has a timestamp of 2.

**Scenario:**

- **T1** and **T2** are both requesting access to data item **X**. However, **T2** holds the lock on **X**.
- Since **T2** is younger (timestamp 2), and **T1** is older (timestamp 1), **T1** is allowed to **wait** for **T2** to release the lock.
- If **T1** had a larger timestamp (say, 3), **T1** would be forced to **die** and restart later.

| Transaction | Timestamp | Request for X | Action   |
|-------------|-----------|---------------|----------|
| T1          | 1         | Yes           | Wait     |
| T2          | 2         | Yes           | Hold (X) |

### Wait/Wound Scheme

- **Wait/Wound** is similar to the **Wait/Die** scheme but with the opposite logic for handling conflicting transactions.

- If the requesting transaction has a smaller timestamp (i.e., it is **older**) than the transaction holding the resource, the requesting transaction **wounds (pre-empt)** the holding transaction by forcing it to release the resource and restart.
- If the requesting transaction has a larger timestamp (i.e., it is **younger**), the requesting transaction **waits**.

**Example:** Consider the same two transactions, **T1** and **T2**, with **T1** having a timestamp of 1 and **T2** having a timestamp of 2.

**Scenario:**

- **T1** and **T2** are both requesting access to data item **X**. However, **T2** holds the lock on **X**.
- Since **T1** is older (timestamp 1), it **wounds T2** and forces **T2** to release **X** and restart.
- If **T1** had a larger timestamp (say, 3), **T1** would be forced to **wait** for **T2** to release the lock.

| Transaction | Timestamp | Request for X | Action     |
|-------------|-----------|---------------|------------|
| T1          | 1         | Yes           | Wound (T2) |
| T2          | 2         | Yes           | Hold (X)   |

### Key Differences between Wait/Die and Wait/Wound

| Aspect              | Wait/Die                         | Wait/Wound                                    |
|---------------------|----------------------------------|-----------------------------------------------|
| Older Transaction   | Wait for the younger transaction | Wound the younger transaction (force restart) |
| Younger Transaction | Dies and restarts later          | Wait for the older transaction                |

### Advantages and Disadvantages

- **Wait/Die Scheme:**
  - **Advantage:** It avoids starvation of older transactions. Older transactions are prioritized, so they are never killed.
  - **Disadvantage:** Can lead to more restarts for younger transactions if the system has many younger transactions, possibly increasing system overhead.
- **Wait/Wound Scheme:**
  - **Advantage:** Older transactions can pre-empt younger ones, preventing younger transactions from holding up the system.
  - **Disadvantage:** Younger transactions might have to wait longer or be delayed, which may cause system delays or increased waiting times.

## Concurrency Control with Optimistic Methods in DBMS

In a **concurrent system**, many transactions may attempt to access and modify shared data simultaneously. The goal of concurrency control is to ensure that the transactions do not interfere with each other, leading to **data inconsistency** or **anomalies**. One way to handle concurrency control is through **optimistic concurrency control (OCC)**, which assumes that conflicts between transactions are rare and that transactions can be executed independently most of the time. Only at the end of the transaction is the **conflict detection** performed.

Optimistic concurrency control operates in **three phases**:

1. **Read Phase** (Transaction Execution Phase)
2. **Validation Phase**
3. **Write Phase**

## 1. Read Phase

In this phase, transactions execute their operations (like reads and computations) without worrying about conflicts. The transaction doesn't acquire any locks or resources, allowing other transactions to proceed concurrently.

## 2. Validation Phase

Once a transaction finishes its execution and is ready to commit, it enters the **validation phase**. During this phase, the system checks whether any conflicts have occurred between the current transaction and other transactions that might have been executed concurrently.

The system typically checks whether:

- The **transaction read** any data that was modified by other concurrent transactions.
- The **transaction modified** any data that other transactions might have read or written.

If the transaction's operations conflict with those of other transactions, the system rejects the transaction, and it must be rolled back. If there is no conflict, the transaction is validated and proceeds to the write phase.

## 3. Write Phase

In the final phase, after the transaction is validated, it **commits** and applies its changes to the database.

If a conflict is detected during the **validation phase**, the transaction may be **rolled back** and forced to restart from the **beginning**, and the process is repeated.

### Example of Optimistic Concurrency Control

Let's say we have two transactions, **T1** and **T2**, and they both want to update a shared data item, **X**.

- **T1** starts and reads the current value of **X**.
- **T2** starts at the same time and also reads the current value of **X**.

Now, both **T1** and **T2** can independently make their changes based on their initial read values, but they haven't committed yet.

#### Scenario 1: No Conflict

- **T1** updates the value of **X** based on its read value, say it increments **X** by 10.
- **T2** also updates **X** based on its read value, say it increments **X** by 20.
- Now, both transactions enter the **validation phase**.
  - If both transactions read the same initial value of **X** (say, **X = 100**), their updates (10 and 20) can be validated without conflict.
  - The **validation phase** detects no conflict since **T1** and **T2** have not modified overlapping data at the same time.
  - Both **T1** and **T2** commit their changes, and the final value of **X** will be **130** (assuming no other operations).

#### Scenario 2: Conflict Detected

- Suppose **T1** updates **X** to **110**, and **T2** tries to update **X** to **120**.
- Both transactions read **X = 100** in the read phase.
- However, when **T2** enters the **validation phase**, it checks that **T1** has already updated **X** to **110** and will **rollback T2**.
- **T2** will restart and re-execute, possibly with a new value for **X**.



## Optimistic Concurrency Control Example with Steps

Let's break down an example with actual steps:

- **Initial Value of X = 100.**

### Transaction T1:

1. **T1** starts, reads **X = 100**.
2. **T1** computes **new X = 100 + 10 = 110**.
3. **T1** enters **validation phase** and finds no conflicts with other transactions, so it proceeds to the **write phase** and commits the new value **X = 110**.

### Transaction T2:

1. **T2** starts, reads **X = 100**.
2. **T2** computes **new X = 100 + 20 = 120**.
3. **T2** enters **validation phase** and finds that **T1** has modified **X** to **110** during **T1's write phase**.
4. Since there is a conflict, **T2** is **rolled back** and will restart with the new value of **X = 110**.

### Transaction T2 Restarts:

1. **T2** starts again, reads **X = 110** (the updated value).
2. **T2** computes **new X = 110 + 20 = 130**.
3. **T2** enters the **validation phase** and finds no further conflicts, so it proceeds to commit the new value **X = 130**.

**Final Value of X = 130.**

## Advantages of Optimistic Concurrency Control

1. **Efficient for Low Conflict Scenarios:** OCC works well when there are relatively few conflicts between transactions because it doesn't require locking resources, and transactions can be executed concurrently without waiting for locks.
2. **Reduced Locking Overhead:** Since transactions don't lock resources, there is less overhead compared to locking-based concurrency control methods like two-phase locking.

## Disadvantages of Optimistic Concurrency Control

1. **High Rollback Rate in High Conflict Scenarios:** If many transactions conflict with each other, OCC can result in a lot of rollbacks, which can lead to wasted computation and slower performance.
2. **Validation Overhead:** The validation phase can be costly because it needs to compare all transaction activities to check for conflicts, which could be expensive for larger systems.

## Database Recovery Management: Transaction Recovery

**Transaction Recovery** is an essential part of a **Database Management System (DBMS)**, ensuring the system can recover from various failures (like system crashes, power outages, or hardware failures) and return to a consistent state. The goal of recovery management is to guarantee the **ACID properties** (Atomicity, Consistency, Isolation, and Durability) of transactions even after a failure.

- **Atomicity** ensures that a transaction is either fully completed or not done at all.

- **Durability** ensures that once a transaction is committed, it persists, even in the event of a failure.

## Types of Failures

1. **Transaction Failure:** A transaction might fail due to logical errors (e.g., violating constraints) or explicit rollback commands.
2. **System Failure:** A system crash occurs, which may involve losing the memory or cache of uncommitted transactions.
3. **Media Failure:** Disk failure leads to corruption or loss of data files.

## Transaction Recovery Techniques

There are several methods of transaction recovery. The most commonly used methods are **Undo** and **Redo**, which are applied using **logs** that store the details of the transactions (committed and uncommitted).

Here are some of the key transaction recovery techniques:

1. **Undo Logging (Rollback):**
  - This technique involves logging changes made by transactions so that if a transaction fails, the system can "undo" or roll back the changes made by that transaction. An undo log records the previous state before a change is committed.
2. **Redo Logging (Forward Recovery):**
  - Redo logging is the process of maintaining a log of changes in a way that allows the system to reapply those changes during recovery. If a transaction is committed and a failure occurs afterward, the changes can be reapplied to restore the database to its last consistent state.
3. **Checkpointing:**
  - A checkpoint is a specific point in time where the database is consistent, and all changes up to that point are written to disk. During recovery, the system can start from the last checkpoint instead of the beginning of the log, thus reducing the amount of log data that needs to be processed.
4. **Shadow Paging:**
  - Shadow paging involves maintaining two pages for each database page: a shadow page (the old copy) and the current page. Transactions modify the current page, and if a failure occurs, the system can revert to the shadow page, effectively "discarding" the uncommitted changes.
5. **Two-Phase Commit Protocol:**
  - In distributed databases, the two-phase commit protocol is employed to ensure all participating databases either commit or roll back a transaction. It consists of a prepare phase (where participants prepare to commit) and a commit phase (where they either commit the transaction or roll back).
6. **Write-Ahead Logging (WAL):**
  - In WAL, logs are written before any changes are applied to the database. This ensures that the log always reflects the state of the database, allowing recovery to be achieved by using the log entries to redo or undo transactions as needed.

## 7. Logging Protocols:

- Various logging protocols manage the order and structure of log entries. Examples include simple log-based protocols, and more advanced versions like ARIES (Algorithm for Recovery and Isolation Exploiting Semantics).

## 8. Consistency Checks:

- After recovery, many systems perform consistency checks to ensure that the database reflects a valid state. This might involve checking data integrity constraints, relationships, and the correct application of transactions.

## Write-Ahead Logging (WAL) Protocol

The **WAL protocol** is the basis for many recovery mechanisms. It states that before any changes made by a transaction are written to the database, a log entry for that transaction must first be written to the **log file**.

The log contains:

- **Before-image:** The state of the data before the transaction modifies it.
- **After-image:** The state of the data after the transaction modifies it.
- **Transaction ID:** Identifies the transaction.
- **Commit or Abort:** Indicates whether the transaction committed or was rolled back.

## Example Scenario: Transaction Recovery

Let's go through a simple example with a log file that records transactions and their effects on a database.

Consider the following scenario:

### Initial Database State:

- **Balance of Account A = 1000**
- **Balance of Account B = 500**

### Transaction T1: Transfer \$200 from Account A to Account B.

1. **T1** starts by reading the balance of **Account A** (1000) and **Account B** (500).
2. **T1** deducts \$200 from **Account A**, so the new balance of **Account A** is 800.
3. **T1** adds \$200 to **Account B**, so the new balance of **Account B** is 700.
4. **T1** commits, meaning it writes the final changes to the database.

### Log Entry for Transaction T1 (before commit):

T1: Before-image: Account A = 1000, Account B = 500

T1: After-image: Account A = 800, Account B = 700

T1: COMMIT

### System Failure during Transaction T1 (before commit):

Suppose that a **system failure** occurs after the transaction **T1** has modified the balances but before it has committed.

## Log Entry for Transaction T1 (before failure):

T1: Before-image: Account A = 1000, Account B = 500

T1: After-image: Account A = 800, Account B = 700

After the failure, the **system recovery** process checks the logs and handles the incomplete transactions.

## Recovery Process

1. **Redo Phase:** If a transaction has committed (i.e., has a COMMIT log entry), the system will **redo** the transaction's changes to ensure that it reflects in the database.
  - In the example, **T1** committed, so the **redo** phase ensures that the changes made by **T1** are written to the database. Account A becomes 800, and Account B becomes 700.
2. **Undo Phase:** If a transaction has not committed (i.e., there is no COMMIT log entry), the system will **undo** the changes made by that transaction.
  - If **T1** had failed before committing, its changes would be undone, meaning **Account A** would return to 1000, and **Account B** would return to 500.

## After Recovery:

- **Account A = 800**
- **Account B = 700**

## Key Concepts in Transaction Recovery

1. **Atomicity:** If a transaction fails before committing, its changes are **undone**.
2. **Durability:** Once a transaction commits, its changes are permanent, even if the system crashes after the commit.

## Example with System Crash before Commit (Undo Example)

Let's consider a scenario where a system crash occurs **before** a transaction commits. Here's how recovery would work:

1. **Transaction T1** starts and makes changes to the database but **does not commit** before the crash.
2. **Log for T1** before crash:

T1: Before-image: Account A = 1000, Account B = 500

T1: After-image: Account A = 800, Account B = 700

During recovery, the system checks the logs:

T1 did not commit, so it needs to be undone.

Account A is reverted to 1000, and Account B is reverted to 500.

Account A = 1000

Account B = 500

In this case, the system ensures that any uncommitted changes are discarded, ensuring atomicity.